UPDATED CODE :

board.py:

```
import random

SIZE = 4


class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()

        self.last_merges = []

    def add_random_tile(self):
        empty = [(r, c) for r in range(SIZE) for c in range(SIZE) if
self.grid[r][c] == 0]
        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2


    @staticmethod

    @staticmethod
    def slide_line_with_merges(line):
        values = [x for x in line if x]
        result = []
        merges = []
        i = 0

        while i < len(values):
            if i + 1 < len(values) and values[i] == values[i + 1]:
                merged_value = values[i] * 2
                result.append(merged_value)
                merges.append(merged_value)
                i += 2
            else:
                result.append(values[i])
                i += 1

        result += [0] * (SIZE - len(result))
        return result, merges

    @staticmethod
```

```
    def slide_line(line):
        result, _ = Board.slide_line_with_merges(line)
        return result




    def move_left(self):
        changed = False
        self.last_merges = []

        for r in range(SIZE):
            old = self.grid[r][:]
            new, merges = self.slide_line_with_merges(old)
            self.grid[r] = new
            self.last_merges.extend(merges)
            changed |= old != new

        return changed

    def move_right(self):
        changed = False
        self.last_merges = []

        for r in range(SIZE):
            old = self.grid[r][:]
            new, merges = self.slide_line_with_merges(list(reversed(old)))
            new = list(reversed(new))
            self.grid[r] = new
            self.last_merges.extend(merges)
            changed |= old != new

        return changed

    def move_up(self):
        changed = False
        self.last_merges = []

        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new, merges = self.slide_line_with_merges(old)

            for r in range(SIZE):
                self.grid[r][c] = new[r]

            self.last_merges.extend(merges)
            changed |= old != new

        return changed
```

```
    def move_down(self):
        changed = False
        self.last_merges = []

        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new, merges = self.slide_line_with_merges(list(reversed(old)))
            new = list(reversed(new))

            for r in range(SIZE):
                self.grid[r][c] = new[r]

            self.last_merges.extend(merges)
            changed |= old != new

        return changed


    def can_move(self):
        if any(0 in row for row in self.grid):
            return True
        for r in range(SIZE):
            for c in range(SIZE):
                if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                    return True
                if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                    return True
        return False
```

game.py

```
from board import Board


class Game:
    def __init__(self):
        self.board = Board()
        self.best_score = 0
        self.history = None  # Stores one previous successful move

    def display(self):
        print("\n" + "+------+------+------+------+")
        for row in self.board.grid:
            print("|" + "|".join(
                f"{x:^6}" if x else f"{' ':^6}" for x in row
```

```
            ) + "|")
            print("+------+------+------+------+")
        print("Score:", self.board.score, " Best:", self.best_score)

    def has_won(self):
        return any(2048 in row for row in self.board.grid)

    def move(self, key):
        moves = {
            "a": self.board.move_left,
            "d": self.board.move_right,
            "w": self.board.move_up,
            "s": self.board.move_down
        }

        if key not in moves:
            return False

        # Save the current state before attempting the move.
        previous_grid = [row[:] for row in self.board.grid]
        previous_score = self.board.score

        changed = moves[key]()

        if not changed:
            # An unchanged move must not replace undo history.
            return False

        # Replace the previous snapshot with this successful move's
        # starting state. This supports exactly one level of undo.
        self.history = (previous_grid, previous_score)

        # Update the best score for this session.
        self.best_score = max(self.best_score, self.board.score)

        # Spawn a tile only after a successful move, unless the game
        # has already been won.
        if not self.has_won():
            self.board.add_random_tile()

        return True

    def undo(self):
        if self.history is None:
            return False

        previous_grid, previous_score = self.history
```

```
        # Restore the board and score without generating a tile.
        self.board.grid = [row[:] for row in previous_grid]
        self.board.score = previous_score

        # Clear history so the same move cannot be undone repeatedly.
        self.history = None

        return True

    def run(self):
        print("2048 — W/A/S/D to move, U to undo, Q to quit.")

        while True:
            self.display()

            if self.has_won():
                print("You reached 2048! You win!")
                return

            if not self.board.can_move():
                print("Game over! No legal moves remain.")
                return

            key = input("> ").strip().lower()

            if key == "q":
                print("Game ended.")
                return

            if key == "u":
                if self.undo():
                    print("Previous move undone.")
                else:
                    print("Nothing to undo.")
                continue

            if key not in ("w", "a", "s", "d"):
                print("Invalid command. Use W/A/S/D, U, or Q.")
                continue


            if self.move(key):
                merges = self.board.last_merges

                if merges:
                    merged_values = ", ".join(map(str, merges))
                    print(
                        f"Move successful. Merged tiles: {merged_values}."
```

```
                )
            else:
                print("Move successful. No tiles merged.")
        else:
            print("No change. Try another direction.")
```